



Programação Orientada a Objetos

Classes e objetos

Maurício da Motta Braga
mmb2@cesar.school



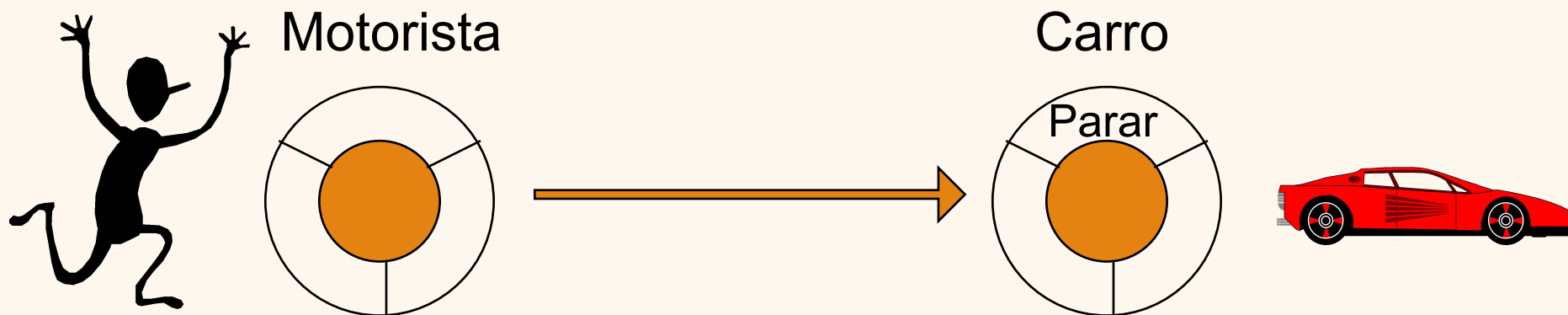


*"Object thinking is not
about the objects. It is
about your mind-set."*

Rebecca Wirfs-Brock

Desenvolvimento Orientado a Objetos

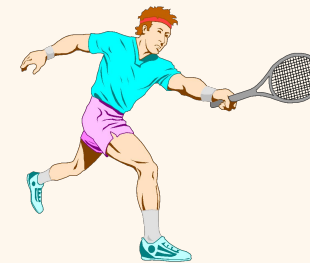
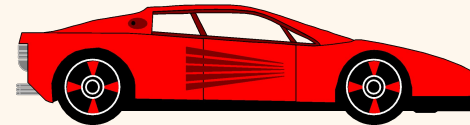
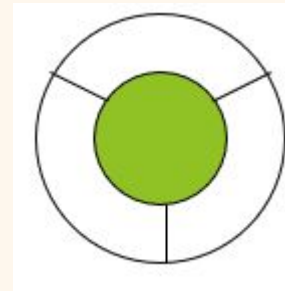
Visualiza e representa o mundo real como um **conjunto de objetos** que interagem entre si para que determinadas operações sejam realizadas.



Desenvolvimento Orientado a Objetos

- ▶ Objetos do mundo real transformam-se em objetos no software.
- ▶ Viabiliza a montagem de sistemas a partir de componentes.

Conta



Objetos

- ▶ Um objeto é qualquer coisa, real ou abstrata, sobre a qual armazenamos dados e operações que manipulam tais dados.
- ▶ Unidade básica de modularização na abordagem OO.
- ▶ Um objeto é um ente independente, composto por:
 - ▶ atributos, isto é, características ou propriedades que definem o objeto
 - ▶ comportamento, isto é, um conjunto de ações pré-definidas (denominada métodos), através das quais o objeto responderá à demanda de processamento por parte de outros objetos

Exemplo

- ▶ Você é contratado para desenvolver um sistema para gerenciar as atividades de uma pizzeria.
- ▶ Existem vários objetos na pizzeria que podem ser representados nesse software:
 - ▶ Pizza, cliente, mesa, garçom, etc....
- ▶ Cada objeto tem características que o descrevem:
 - ▶ Mesa ocupada ou não
 - ▶ Mesa reservada ou não
 - ▶ Quantidade de pessoas que podem utilizá-la etc.



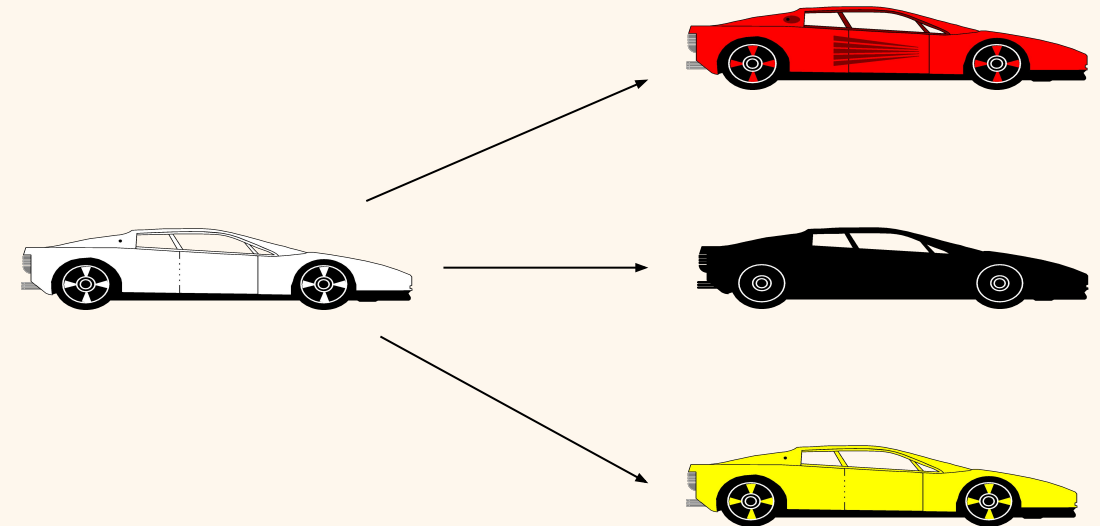
Exercício

- ▶ Para um sistema de gerenciamento de farmácias, que objetos poderiam ser representados nesse software?



Classes

- ▶ Classes: “fábricas” de objetos.
- ▶ Exemplificando:
 - ▶ Pessoa é uma classe e João é um objeto (instância) da classe Pessoa.
 - ▶ Um carro é uma classe; “meu carro” é um objeto.



Conceito de classe

- ▶ Classes englobam dados e procedimentos que descrevem o conteúdo e o **comportamento** de entidades do mundo real, que são representadas por objetos.
- ▶ Imagine o que um objeto irá realizar. Para isso, é preciso verificar três pontos:
 - ▶ Identidade do objeto que vai ser criado. Essa identidade deve ser única dentro do sistema, para que não haja conflito, ou seja, sem repetições.
 - ▶ Características do objeto. Dentro da programação orientada a objetos, as características são nomeadas como propriedades ou **atributos**. Por exemplo, as propriedades de um objeto “pessoa” poderiam ser “peso”, “tamanho” e “idade”.
 - ▶ Ações que o objeto irá executar, denominadas de **métodos**. Eles podem ser muito variados, dependendo do tipo de solução desenvolvida.

Classes em Java

Estrutura de uma classe em Java

```
[acesso] class NomeDaClasse  
{  
    Atributos  
    Construtores  
    Métodos  
}
```

Definição da
classe

O conteúdo da
classe

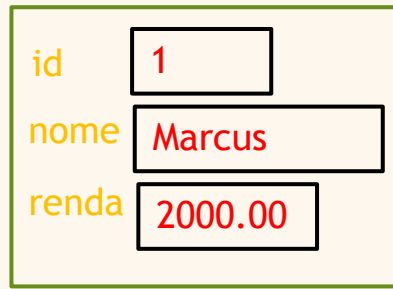
Conceito de classe

- ▶ Classes são “moldes” para objetos, assim como tipos estruturados de dados são “moldes” para registros

```
struct Cliente {  
    int id;  
    char[100] nome;  
    float renda;  
};
```

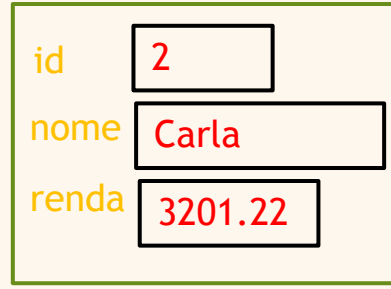
Tipo estruturado de dados

registro



tipo: Cliente

registro

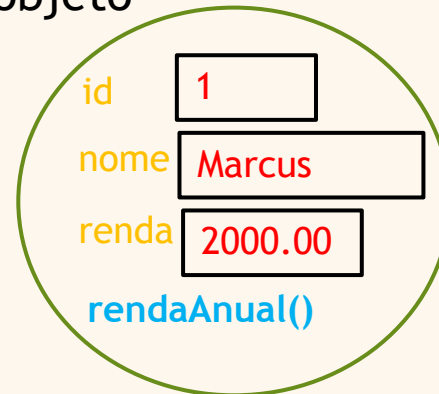


tipo: Cliente

```
class Cliente {  
    int id;  
    String nome;  
    double renda;  
    double rendaAnual() {  
        return renda * 12;  
    }  
}
```

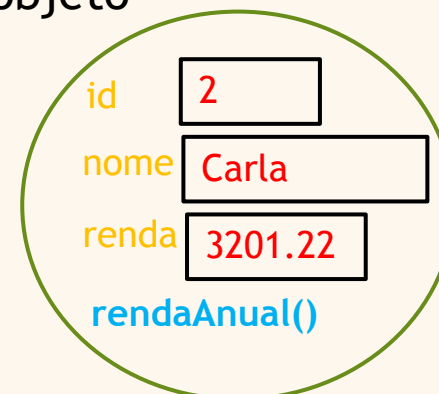
Classe

objeto



tipo: Cliente

objeto



tipo: Cliente

Exemplo

```
public class Produto {  
  
    private int  codigo;  
    private String  nome;  
  
    public int getCodigo() {  
        return codigo;  
    }  
    public void setCodigo(int cod) {  
        codigo = nome_produto;  
    }  
    public String getNome() {  
        return nome;  
    }  
    public void setNome(String nome_produto) {  
        nome = nome_produto;  
    }  
}
```

- The diagram shows three labels at the top: "Declaração do tipo da variável", "Variável", and "Objeto criado". Below them is the code snippet: `Produto p = new Produto();`. Arrows point from each label to its corresponding part of the code: from "Declaração do tipo da variável" to `Produto`, from "Variável" to `p`, and from "Objeto criado" to `new Produto()`.

Construtores

- ▶ Inicializam um objeto.
- ▶ Possuem o mesmo nome das suas classes.
- ▶ Costumam receber valores de parâmetros externos para definir os atributos.

EX:

```
public Produto(int cod, String nome_prod)
{
    codigo = cod;
    nome = nome_prod;
}
```

```
int cod = 341278;
String prod = "Computador MSX Expert";

Produto p = new Produto(cod, prod);
```

Criação de objetos a partir de uma classe

Relação entre variável e objeto

- ▶ O “acesso” ao objeto se dá normalmente através de uma variável do tipo da classe originária do objeto
- ▶ Esta variável armazena o endereço de memória do objeto criado
- ▶ Portanto, diz-se que a variável aponta para o objeto, ou seja, é uma referência para o objeto
- ▶ Uma variável de tipo não primitiva pode não apontar para objeto nenhum. Neste caso, diz-se que ela é `null`, uma palavra reservada
- ▶ Um mesmo objeto pode ser referenciado por mais de uma variável

Construtores

- ▶ Pode existir mais de um construtor em uma classe;
- ▶ Caso não seja definido nenhum construtor, o compilador cria o construtor padrão, sem parâmetros;
 - ▶ O construtor padrão inicializa os atributos de classe para seus valores padrões.
- ▶ Caso seja definido qualquer construtor, o construtor padrão não será adicionado pelo compilador;
 - ▶ Caso se tenha interesse que a classe ainda possua um construtor sem parâmetros, é necessário declará-lo explicitamente



A Referência **this**

- ▶ Métodos de instância recebem um argumento chamado **this** que é uma referência ao objeto corrente.
- ▶ O uso explícito do **this** é necessário quando:
 - ▶ O nome de um parâmetro do método for o mesmo de uma variável de instância;
 - ▶ Desejamos passar uma referência do objeto corrente como parâmetro para outro método.

EX:

```
Class Produto {  
  
    private int codigo;  
    private String nome;  
  
    public Produto(int codigo, String nome)  
    {  
        this.codigo = codigo;  
        this.nome = nome;  
    }  
}
```

Criação de objetos a partir de uma classe

Métodos – passagem de parâmetros

- ▶ JAVA só suporta passagem de parâmetros por valor (por cópia)
- ▶ O escopo dos parâmetros de um método é o corpo do método
- ▶ Parâmetros de tipos primitivos: copia-se para a variável que define o parâmetro o valor da variável ou da constante passada
- ▶ Parâmetros de tipos não primitivos: copia-se para a variável que define o parâmetro a referência (endereço de memória) passada

Criação de objetos a partir de uma classe

Efeitos na memória em tempo de execução

- ▶ Criam-se variáveis e objetos na memória
- ▶ Uma variável sempre aponta para um único objeto
- ▶ Um objeto pode ser referenciado por mais de uma variável



Anotações

- ▶ São criadas usando o símbolo arroba (@) seguido do nome da mesma e opcionalmente, parâmetros. Podemos criar nossas próprias anotações ou utilizar as anotações predefinidas do Java e dos frameworks.
- ▶ Útil quando desejamos anotar código de maneira que essas marcações possam ser verificadas em tempo de compilação, ou utilizadas por ferramentas tais como frameworks de desenvolvimento de software.
- ▶ Existem três categorias de anotações.
 - ▶ Anotações marcadoras;
 - ▶ Anotações de valor único;
 - ▶ Anotações completas.



Anotações marcadoras

- ▶ São identificadas apenas pelo nome, sem dados adicionais.
- ▶ Exemplo: anotações padrão do Java (podem ser utilizadas sem ser importadas), tais como `@Override` (sinaliza sobreescrita de método) e `@Deprecated` (sinaliza método obsoleto, que não deve mais ser utilizado).



Anotações de valor único

- ▶ São similares às anotações marcadoras, no entanto, possuem um único membro, chamado valor.
- ▶ São representadas pelo nome da anotação e um par nome=valor, ou simplesmente com o valor, entre parênteses.
- ▶ Sintaxe: @MinhaAnotacao("valor")



Anotações completas

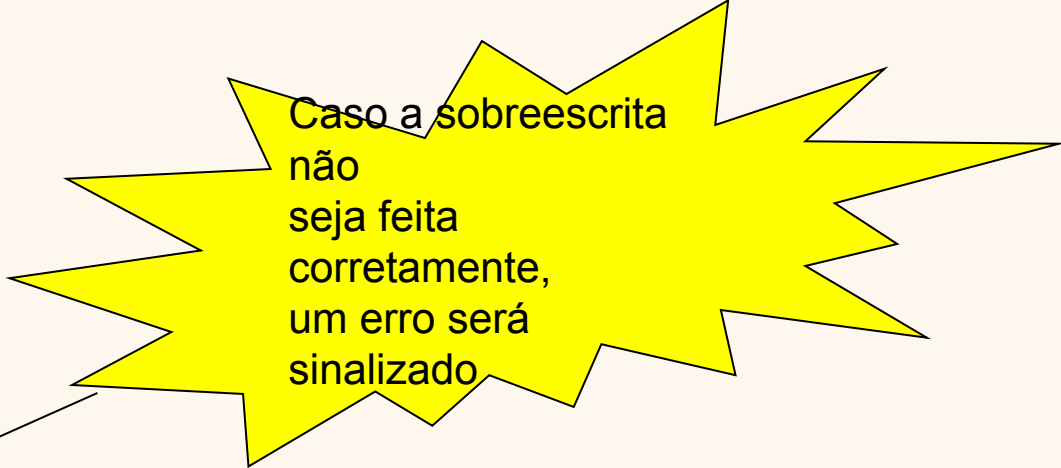
- ▶ São aquelas que possuem múltiplos membros. Portanto, neste tipo, devemos usar a sintaxe completa para cada par nome=valor.
 - ▶ Cada par é informado separado do outro por uma vírgula.
 - ▶ Exemplo: `@Version(major=1, minor=0, micro=0)`.
-

Exemplo com anotação Deprecated

```
public class Exemplo {  
  
    @Deprecated  
    public void metodoFazAlgo() {  
        // ...  
    }  
  
    public void metodoFazAlgoMelhorado() {  
        // ...  
    }  
}
```

Exemplo com anotação Override

```
class Funcionario {  
    protected double salario;  
    public double getSalario(double bonus) {  
        return this.salario + bonus;  
    }  
}  
  
class FuncionarioComissionado extends Funcionario {  
    protected double extra;  
    @Override  
    public double getSalario(float bonus) {  
        return this.salario + this.extra + bonus;  
    }  
}
```



Caso a sobreescrita
não
seja feita
corretamente,
um erro será
sinalizado

Exemplo Spring com múltiplas anotações

```
package com.exemplo.cadastroclientes.model;  
import jakarta.persistence.*;  
import jakarta.validation.constraints.*;
```

@Entity

```
public class Cliente {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

@NotBlank(message = "Nome é obrigatório")

@Size(min = 3, max = 50, message = "Nome deve ter entre 3 e 50 caracteres")

```
private String nome;
```

@NotBlank(message = "E-mail é obrigatório")

@Email(message = "E-mail inválido")

```
private String email;
```

```
// métodos e construtor omitidos.
```



**Slides adaptados a partir
do material produzido pelo
prof. Eduardo Calábria**

egc2@cesar.school





**Maurício da Motta
Braga**



mmb2@cesar.school

